Unit Notes: NoSQL Databases and Big Data Storage Systems

1. What is Big Data? — Setting the Context

Before understanding NoSQL, it is essential to understand why it was needed in the first place.

Traditional data was small, structured, and neatly stored in spreadsheet-like tables. But today, enormous amounts of data are generated every second — from social media posts, online shopping, GPS tracking, sensor devices, and video streaming. This is called **Big Data**.

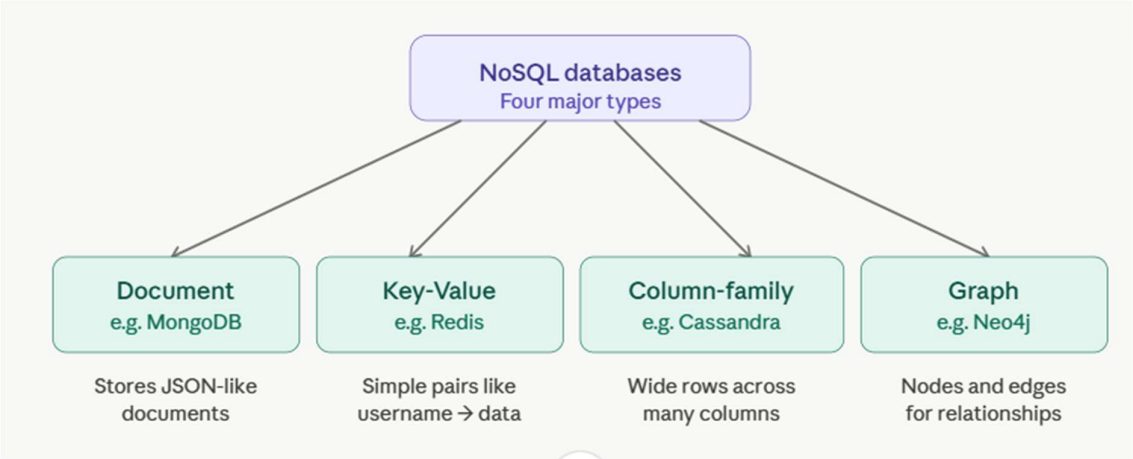
Big Data is characterized by the **5 V's:**

- Volume — massive amounts of data (petabytes and beyond)
- Velocity — data generated at very high speed in real time
- Variety — data comes in many formats (text, images, videos, logs)
- Veracity — uncertainty about data accuracy and quality
- Value — extracting useful meaning from raw data

Traditional relational databases (like MySQL or Oracle) were not designed to handle this scale. They struggle with unstructured data, require a fixed schema, and cannot easily be distributed across thousands of servers. This gap gave rise to **NoSQL databases**.

2. Introduction to NoSQL Systems

NoSQL stands for "Not Only SQL." It is a broad category of database systems designed to store, retrieve, and manage data that does not fit neatly into relational tables.



2.1 Why Was NoSQL Created?

Imagine a school that previously stored student records in neat spreadsheet tables. Each student had a fixed set of columns: name, roll number, marks in 5 subjects. This worked fine for 200 students. Now imagine tracking 2 million students globally, where each student has a different set of data — some have 10 subjects, some have projects, some have activity records, and some have multimedia portfolios. A rigid table no longer works. NoSQL was created to solve exactly this kind of problem.

2.2 Key Characteristics of NoSQL Systems

NoSQL databases share several important properties that distinguish them from traditional SQL systems:

Schema-free design means data does not need to follow a rigid predefined structure. You can add new fields to a document at any time without altering a table definition. This is particularly useful when the data format evolves over time, such as in product catalogues where new product attributes are added regularly.

Horizontal scalability means that instead of buying a bigger, more powerful single server (vertical scaling), NoSQL systems can spread data across hundreds or thousands of ordinary servers (horizontal scaling). This is how companies like Amazon and Facebook handle billions of records.

High availability is achieved through data replication — copies of data are stored on multiple servers so that if one server fails, another takes over automatically without any downtime.

Eventual consistency is a trade-off adopted by many NoSQL systems. Rather than guaranteeing that all servers have the same data at the exact same moment (which is expensive and slow), they guarantee that all copies will eventually become consistent. This is acceptable for applications like social media feeds but not for bank transactions.

2.3 The CAP Theorem (Beginner Explanation)

The CAP theorem states that any distributed data system can guarantee at most two of the following three properties simultaneously:

- **Consistency** — every read receives the most recent write

- **Availability** — every request receives a response (not necessarily the most current data)
- **Partition tolerance** — the system continues to operate even if network communication between servers breaks

NoSQL databases generally prioritize availability and partition tolerance (AP systems), while traditional SQL databases prioritize consistency and availability (CA systems). Understanding this helps you choose the right database for a given problem.

JSON → Used for **data sharing and readability**
BSON → Used for **efficient storage and processing** in MongoDB

3. Types of NoSQL Databases

3.1 Document-Based (e.g., MongoDB)

Stores data as documents — typically in JSON or BSON format. Each document is a self-contained unit that can have a different structure from other documents. This is the most beginner-friendly NoSQL type and the focus of this topic.

Real-world analogy: Think of a document store like a filing cabinet where each folder (document) can contain whatever papers (fields) are relevant to that particular case — no two folders need to look the same.

3.2 Key-Value Store (e.g., Redis, DynamoDB)

Stores data as simple pairs — a unique key and its associated value. It is the simplest NoSQL type and extremely fast for lookup operations.

Real-world analogy: Like a locker room — each locker has a number (key) and stores whatever the person puts in it (value). You can retrieve contents instantly if you know the locker number.

3.3 Column-Family Store (e.g., Apache Cassandra, HBase)

Stores data in columns rather than rows, which makes it efficient for reading specific attributes across a large dataset. Used heavily in analytics and time-series data.

Real-world analogy: Instead of storing a student's complete record in one row, imagine storing all students' marks in one column and all addresses in another column. Queries that need only marks never have to load address data.

3.4 Graph Database (e.g., Neo4j)

Stores data as nodes (entities) and edges (relationships between entities). Best suited for highly connected data like social networks, recommendation engines, or fraud detection.

Real-world analogy: Like a map of roads — cities are nodes and roads connecting them are edges. Finding the shortest path between two cities is a graph problem.

4. Document-Based NoSQL Systems and MongoDB

4.1 What is a Document Database?

A document database stores each record as a **document** — a structured, self-contained unit of data. Documents are typically written in **JSON (JavaScript Object Notation)** format, which uses key-value pairs enclosed in curly braces.

Here is what a student record looks like in a document database compared to a relational table:

Relational table row:

roll_no	name	subject1	subject2	subject3
101	Ravi	85	90	78

Document database record:

json

```
{
  "roll_no": 101,
  "name": "Ravi Kumar",
  "subjects": [
    { "name": "DBMS", "marks": 85 },
    { "name": "OS", "marks": 90 },
    { "name": "Networks", "marks": 78 }
  ],
  "address": {
    "city": "Guntur",
    "state": "Andhra Pradesh"
  },
  "hobbies": ["chess", "cricket"]
}
```

Notice that the document stores nested objects (address) and arrays (subjects, hobbies) — all within a single record. A relational table cannot represent this without multiple separate tables and JOIN operations.

4.2 Introduction to MongoDB

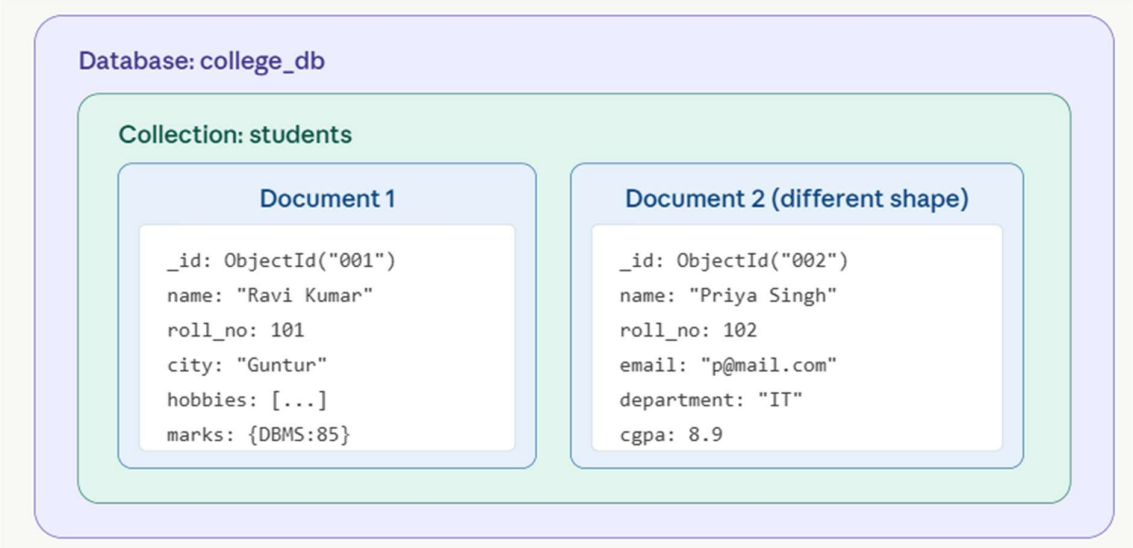
MongoDB is the most widely used document database in the world. It was created in 2007 and its name comes from the word "humongous," reflecting its ability to handle enormous datasets.

MongoDB stores data in a format called **BSON (Binary JSON)**, which is similar to JSON but supports additional data types like Date, ObjectId, and binary data. From the developer's perspective, it looks and behaves exactly like JSON.

Key MongoDB terminology:

MongoDB Term	SQL Equivalent	Meaning
Database	Database	Container for all collections
Collection	Table	Group of related documents
Document	Row / Record	One unit of data
Field	Column	A key-value pair within a document
_id	Primary Key	Unique identifier, auto-generated by MongoDB

4.3 MongoDB Architecture



visualize show_widget

Notice that Document 1 and Document 2 have different fields — Document 2 has email, department, and cgpa fields that Document 1 does not. This **flexible schema** is the core advantage of MongoDB. In a relational database, all rows must have the same columns, meaning you would need to add NULL values for missing data or alter the table structure.

4.4 Data Types Supported in MongoDB

MongoDB supports a rich variety of data types within a single document:

- **String** — "name": "Ravi Kumar" — textual data
- **Number (Integer / Double)** — "age": 21, "cgpa": 8.75
- **Boolean** — "is_active": true
- **Array** — "hobbies": ["chess", "cricket", "coding"]
- **Embedded Document (Object)** — "address": { "city": "Guntur", "pincode": "522001" }
- **Date** — "dob": ISODate("2003-05-15")
- **ObjectId** — "_id": ObjectId("64a1...") — auto-generated unique identifier
- **Null** — "middle_name": null

4.5 CRUD Operations in MongoDB

CRUD stands for Create, Read, Update, and Delete — the four fundamental database operations. In MongoDB, these are performed using simple JavaScript-like method calls.

Create — Inserting Documents
javascript

```
// Insert a single document
```

```javascript
db.students.insertOne({
  name: "Ravi Kumar",
  roll_no: 101,
  department: "IT",
  cgpa: 8.5,
  city: "Guntur"
})

// Insert multiple documents at once
db.students.insertMany([
  { name: "Priya Singh", roll_no: 102, department: "CSE", cgpa: 9.1 },
  { name: "Arjun Rao",   roll_no: 103, department: "IT",  cgpa: 7.8 }
])
```

Read — Querying Documents

javascript

```javascript
// Get ALL documents in the collection
db.students.find()

// Get documents matching a condition (WHERE clause equivalent)
db.students.find({ department: "IT" })

// Get students with cgpa greater than 8
db.students.find({ cgpa: { $gt: 8 } })

// Get only name and cgpa fields (projection — like SELECT in SQL)
db.students.find({ department: "IT" }, { name: 1, cgpa: 1, _id: 0 })

// Get the first matching document only
db.students.findOne({ roll_no: 101 })

// Sorting results (1 = ascending, -1 = descending)
db.students.find().sort({ cgpa: -1 })

// Limit results to top 5
db.students.find().sort({ cgpa: -1 }).limit(5)
```

Update — Modifying Documents

javascript

```javascript
// Update one document — set a new value
db.students.updateOne(
  { roll_no: 101 },          // filter: find this document
  { $set: { cgpa: 8.9 } }    // update: change cgpa
)

// Update all IT students — add a new field
db.students.updateMany(
```

```
  { department: "IT" },
  { $set: { lab_batch: "B3" } }
)

// Increment a numeric field (e.g., add 5 to marks)
db.students.updateOne(
  { roll_no: 101 },
  { $inc: { "marks.DBMS": 5 } }
)
```

Delete — Removing Documents

javascript

```
// Delete one document
db.students.deleteOne({ roll_no: 101 })

// Delete all documents matching a condition
db.students.deleteMany({ department: "CA" })

// Delete ALL documents in a collection (use with caution)
db.students.deleteMany({})
```

4.6 Common Query Operators in MongoDB

MongoDB provides comparison and logical operators similar to SQL's WHERE clause conditions:

Operator	Meaning	Example
$eq	Equal to	{ age: { $eq: 21 } }
$ne	Not equal to	{ dept: { $ne: "CA" } }
$gt	Greater than	{ cgpa: { $gt: 8 } }
$gte	Greater than or equal	{ marks: { $gte: 60 } }
$lt	Less than	{ fees: { $lt: 50000 } }
$lte	Less than or equal	{ age: { $lte: 25 } }
$in	Value exists in a list	{ dept: { $in: ["IT","CSE"] } }
$and	All conditions must match	{ $and: [{cgpa:{$gt:8}}, {dept:"IT"}] }
$or	At least one condition matches	{ $or: [{city:"Guntur"},{city:"Vijayawada"}] }

4.7 Indexing in MongoDB

An **index** is a special data structure that speeds up query performance. Without an index, MongoDB must scan every document in a collection to find matching records — this is called a **collection scan** and is very slow on large datasets.

Think of an index like the index at the back of a textbook. Instead of reading every page to find "Binary Search," you look it up in the index and jump directly to the relevant pages.

javascript

```
// Create an index on the "department" field
db.students.createIndex({ department: 1 })

// Create a compound index on two fields
db.students.createIndex({ department: 1, cgpa: -1 })

// View all indexes on a collection
db.students.getIndexes()
```

The _id field is automatically indexed in every MongoDB collection.

4.8 Aggregation in MongoDB

Aggregation is used for advanced data analysis — grouping, counting, averaging, and summarizing data. It works like a pipeline where data passes through multiple stages.

javascript

```
// Example: Count students per department and calculate average CGPA
db.students.aggregate([
  {
    $group: {
      _id: "$department",          // group by department
      total_students: { $sum: 1 }, // count students in each group
      avg_cgpa: { $avg: "$cgpa" }  // calculate average CGPA
    }
  },
  { $sort: { avg_cgpa: -1 } }      // sort by average CGPA descending
])
```

Common aggregation stages:

Stage	Purpose
$match	Filter documents (like WHERE)
$group	Group documents and compute values
$sort	Sort the results
$project	Select or reshape fields
$limit	Restrict number of output documents
$count	Count total matching documents

5. Advantages and Limitations of MongoDB

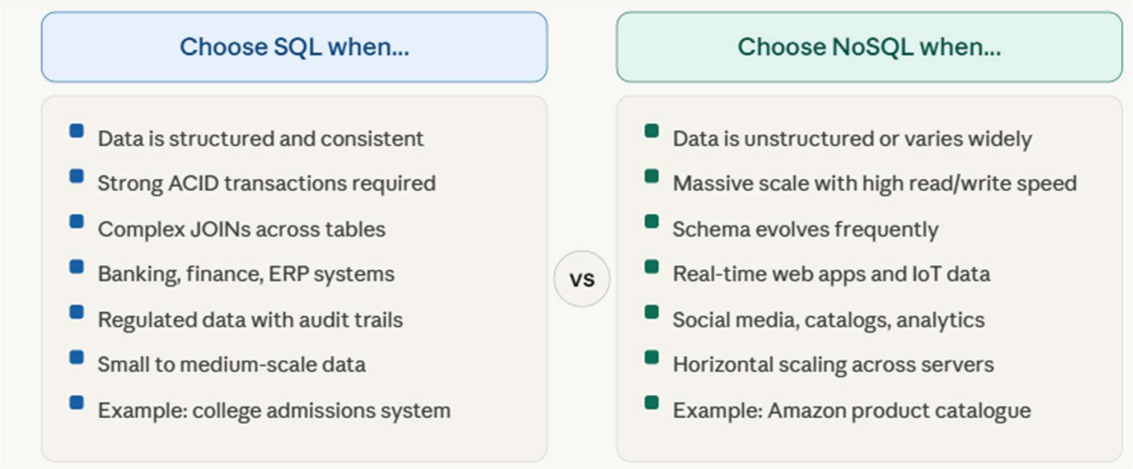
Advantages

MongoDB's flexible schema allows development teams to change the data model rapidly as application requirements evolve — no database migrations needed. Its native support for JSON makes it a natural fit for JavaScript-based web applications. Horizontal scaling through sharding allows it to handle datasets that far exceed what a single server can store. It also supports rich querying, aggregation, and full-text search out of the box.

Limitations

MongoDB is not ideal for highly relational data with complex multi-table JOIN requirements, since it does not support JOINs natively (though it does provide a $lookup operator for basic cross-collection queries). It also consumes more storage than relational databases because data is often duplicated across documents rather than normalized. It is not recommended for applications requiring strict ACID transactions across multiple documents, although recent versions of MongoDB have added multi-document transaction support.

6. When to Use SQL vs NoSQL — Quick Decision Guide



7. Summary — Key Points to Remember

The following are the essential concepts covered in this topic, formatted for quick revision:

NoSQL basics: NoSQL (Not Only SQL) is a family of database systems designed for flexible, scalable, and high-performance storage of modern data. It was born out of the limitations of relational databases in handling Big Data.

Four types: The four major NoSQL types are Document (MongoDB), Key-Value (Redis), Column-Family (Cassandra), and Graph (Neo4j). Each is optimized for a different data access pattern.

MongoDB fundamentals: MongoDB stores data as BSON documents in collections inside a database. Each document can have a unique structure (flexible schema), and every document is automatically assigned a unique _id field.

CRUD operations: The four core operations are insertOne/insertMany for creating data, find/findOne for reading, updateOne/updateMany for modifying, and deleteOne/deleteMany for removing documents.

Query operators: MongoDB uses operators like $gt, $lt, $eq, $in, $and, and $or to filter documents during queries — equivalent to SQL's WHERE clause conditions.

Indexing: Indexes dramatically speed up query performance by creating a lookup structure on specific fields. Without indexes, MongoDB performs a full collection scan, which is slow on large datasets.

Aggregation: The aggregation pipeline allows complex data analysis — grouping, counting, averaging, and sorting — using stages like $match, $group, $sort, and $project.

CAP theorem: Distributed databases cannot simultaneously guarantee all three of Consistency, Availability, and Partition tolerance. Most NoSQL systems choose availability and partition tolerance.

What is a Relational Database?

A Relational Database is a type of database that organizes data into tables with rows and columns. These tables are related to each other through common fields, hence the name “relational.” This structure helps in efficient storage and retrieval of data and also in maintaining data integrity.

Imagine a library catalog system as an example. In a Relational Database, you might have separate tables for books, authors, and borrowers. The **“books” table** could contain columns like *book ID, title, publication date, and author ID.* The **“authors” table** has columns for *author ID, first name, and last name.* The **“borrowers” table** include *borrower ID, name, and contact information.*

These tables are connected through **relationships.** For example, the *author ID* in the **books table** would link to the corresponding *author ID* in the **authors table.** This way, you can easily find all books written by a specific author or get the author’s details for a particular book.

Relational Databases use a language called SQL (Structured Query Language) to manage and query data. SQL lets users to perform complex operations like joining tables, filtering data, and aggregating results.

For example, you can use SQL to find all books published after 2023 by authors from a specific country.

Some popular Relational Database Management Systems (RDBMS) include MySQL, PostgreSQL, Oracle, and Microsoft SQL Server. These systems are widely used in various applications, from small business management tools to large-scale enterprise systems.

Advantages of Relational Databases

Relational Databases offer several key advantages that have made them the go-to choice for many applications:

Relational Databases enforce **data integrity** through various constraints. For example, primary keys ensure each record is unique, while foreign keys maintain relationships between tables. This helps to prevent data duplication and ensures consistency across the database.

Most Relational Databases follow **ACID principles** (Atomicity, Consistency, Isolation, Durability). This ensures that database transactions are processed reliably. For example, in a banking system, when transferring money between accounts, ACID properties ensure that the money is both deducted from one account and added to another, or neither operation occurs.

Relational Databases are better at **handling structured data** with clear relationships. They're ideal for applications where data has a predictable schema, such as financial systems, inventory management, or customer relationship management (CRM) systems.

SQL provides a **standardized way to query data**, which make it easy to retrieve complex information. For example, in an e-commerce system, you can write a single SQL query to find the top 10 customers who have spent the most money on electronics in the last six months.

Relational Databases allow **data normalization**, which reduces data redundancy and improves data integrity. For example, in a school management system, student information is stored once in a students table, rather than being repeated in every record related to that student.

Vertical **scalability** (adding more resources to a single server) is typically easier in Relational Databases, many modern RDBMS also support horizontal scalability through features like sharding and replication.

Disadvantages of Relational Databases

Despite their many advantages, Relational Databases also have some limitations:

Relational Databases require a **predefined schema,** which can be difficault to modify once data is inserted. If you need to add a new field to a table with millions of records, it can be a time-consuming and resource-intensive process.

Relational Databases can scale vertically quite well, **horizontal scaling** (across multiple servers) can be complex and expensive, especially for very large datasets.

As data volumes grow into the terabytes or petabytes, Relational Databases may **struggle with performance**, particularly for read-heavy workloads.

Relational Databases are **not ideal for storing and querying unstructured** or semi-structured data, such as social media posts, sensor data, or complex nested objects.

There can be a disconnect between the way data is represented in object-oriented programming languages and how it's stored in tables and cause **complexity** in application development.

What is NoSQL?

NoSQL, which stands for "Not Only SQL," refers to a category of databases that provide a mechanism for storage and retrieval of data that is modeled differently from the tabular relations used in Relational Databases. NoSQL databases are designed to handle large volumes of unstructured or semi-structured data, offer high performance, and easily scale horizontally.

Unlike Relational Databases, NoSQL databases don't adhere to a fixed schema. They allow for more flexible data models, which can be particularly useful when dealing with evolving data requirements or when the nature of the data doesn't fit neatly into tables.

See also: Are NoSQL Relational Databases?

There are several types of NoSQL databases, each suited to different use cases:

1. **Document Stores:** These databases store data in flexible, JSON-like documents. Each document can have a different structure, allowing for easy storage of complex, hierarchical data. MongoDB is a popular example of a document store.

2. **Key-Value Stores:** These simple databases store data as key-value pairs, similar to a dictionary. They're great for caching and storing user session data. Redis is a well-known key-value store.
3. **Column-Family Stores:** These databases store data in column families, which are containers for rows. They're efficient for queries over large datasets and are often used in big data applications. Cassandra is an example of a column-family store.
4. **Graph Databases:** These databases use graph structures to represent and store data, with nodes, edges, and properties. They're ideal for data with complex relationships, such as social networks or recommendation engines. Neo4j is a popular graph database.

NoSQL databases often use their own query languages or APIs for data manipulation, though some also support SQL-like querying.

For example, MongoDB uses a query language based on JavaScript, while Cassandra has its own query language called CQL (Cassandra Query Language) that's similar to SQL.

Advantages of NoSQL Databases

NoSQL databases has several advantages that make them attractive for certain types of applications:

NoSQL databases **can handle unstructured** and semi-structured data easily. For example, a social media application could use a document store to save user posts, where each post might have a different structure (text, images, videos, etc.).

Many NoSQL databases are designed to **scale horizontally,** which make it easier to handle large volumes of data and high traffic loads. For example, a global e-commerce platform could use a NoSQL database to manage product catalogs across multiple data centers.

NoSQL databases can offer **better performance** for certain operations, especially when dealing with large amounts of data. A real-time analytics system processing millions of events per second might use a column-family store for efficient data ingestion and querying.

NoSQL databases don't require a **fixed schema**, permits easy changes to the data model. This is particularly useful in agile development environments where requirements change frequently.

Some NoSQL databases, particularly document stores, allow developers to store data in a way that's closer to how it's used in application code and reduce the **object-relational impedance mismatch.**

Different types of NoSQL databases are optimized for specific use cases. For example, graph databases excel at managing highly interconnected data, making them ideal for applications like fraud detection systems or recommendation engines.

Disadvantages of NoSQL Databases

Besides benefits, NoSQL databases have some limitations:

Unlike SQL for Relational Databases, there's **no standard query language** across NoSQL databases. Each database might have its own query language or API, which can lead to a steeper learning curve and potential vendor lock-in.

Many NoSQL databases **sacrifice strong consistency** for performance and availability. They often provide eventual consistency, which means that data changes may not be immediately visible across all nodes in a distributed system.

NoSQL databases often **don't support joins** or have limited join capabilities compared to Relational Databases. This can lead to data duplication or require multiple queries to retrieve related data.

Growing rapidly, the **ecosystem around NoSQL** databases (including tools, best practices, and community support) is generally less mature than that of Relational Databases.

The flexible, denormalized data models often used in NoSQL databases can cause **data redundancy**, which may cause consistency issues if not managed.

Many NoSQL databases **don't support ACID transactions** across multiple operations or documents, which can make it difficult to implement certain types of applications, such as financial systems.

Choosing Between Relational and NoSQL Databases

The choice between a Relational Database and a NoSQL database depends on various factors related to your specific use case. Here are some guidelines to help you decide:

You can choose a Relational Database when:

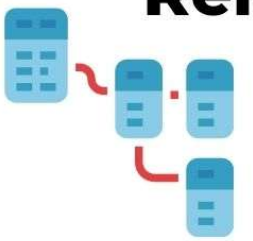
- Your data is structured and relationships between entities are clear and important
- You need strong consistency and ACID compliance (e.g., financial transactions)
- Complex queries and joins are common in your application
- You're building a system where data integrity is crucial (e.g., healthcare systems)
- Your data model is relatively stable and unlikely to change frequently

You can choose a NoSQL Database when:

- You're dealing with large volumes of unstructured or semi-structured data
- Your application needs to handle high velocity data (e.g., real-time analytics)
- You need horizontal scalability for big data
- Your data model is likely to evolve rapidly
- Your application prioritizes availability and partition tolerance over strong consistency
- You're building systems that require low-latency data access at a very large scale

It's pertinent to mention here that many modern applications use a combination of Relational and NoSQL databases and leverage the benefits of each for different aspects of the system. This approach is often referred to as **"polyglot persistence."**

For example, an e-commerce platform may use a Relational Database to handle order processing and inventory management, where data consistency and complex transactions are crucial. At the same time, it might use a NoSQL database to store customer browsing history and product recommendations, where the ability to handle high volumes of rapidly changing data is more important.



Relational Database VS NoSQL

DatabaseTown.com

Criteria	Relational Database	NoSQL Database
Data Model	Tabular (tables with rows and columns)	Varies (document, key-value, column-family, graph)
Schema	Fixed schema	Flexible, schema-less
Scalability	Vertical scalability is easier	Horizontal scalability is easier
ACID Compliance	Typically fully ACID compliant	Varies (some offer eventual consistency)
Query Language	SQL (standardized)	Database-specific languages or APIs
Relationships	Handled through table joins	Often denormalized or handled in application code
Data Integrity	Strong (enforced through constraints)	Varies (often relaxed for performance)
Consistency	Strong consistency	Often eventual consistency (but varies)
Use Cases	Financial systems, CRM, ERP	Big data, real-time web apps, content management
Performance	Excellent for complex queries	Excellent for read/write-heavy workloads
Flexibility	Less flexible	Highly flexible
Transactions	Support for complex transactions	Limited transaction support in many cases
Data Size	Typically smaller to medium-sized datasets	Can handle very large datasets efficiently
Standardization	High (SQL is standardized)	Low (each database may have unique features)

DatabaseTown.com

Relational Database VS NoSQL

Conclusion

Both Relational Databases and NoSQL databases have their place in modern data management. Relational Databases is best suited in the scenarios requiring complex queries, transactions, and strong data integrity. They remain the backbone of many critical systems where structured data and reliable relationships are key.

NoSQL databases, on the other hand, provide solutions for handling large volumes of diverse, rapidly changing data. They offer flexibility and scalability that can be crucial for certain types of applications, particularly those dealing with unstructured data or requiring high performance at massive scale.

Many Relational Databases are adding support for JSON and other semi-structured data types, whereas some NoSQL databases are introducing stronger consistency models and improved support for complex queries.

Ultimately, the choice between Relational and NoSQL databases is dependent on your specific requirements. Consider factors such as the nature of your data, your scalability needs, the types of queries you'll be performing, and your consistency requirements. In many cases, a hybrid approach utilizing both types of databases may provide the best solution.

a) Key Differences: Relational vs NoSQL Databases Key elaborations on the differences:

Data Structure: Relational databases store data in structured tables with predefined rows and columns, enforcing a rigid schema. NoSQL databases support flexible formats — MongoDB uses JSON-like documents, Redis uses key-value pairs, Neo4j uses graph nodes and edges, and Cassandra uses wide columns.

ACID vs BASE: Relational systems guarantee ACID properties, making them ideal for financial or transactional workloads. Most NoSQL systems follow the BASE model (Basically Available, Soft state, Eventual consistency), prioritizing availability and performance over strict consistency.

Scalability: SQL databases scale vertically by upgrading server hardware. NoSQL databases are designed for horizontal scaling — distributing data across multiple commodity servers (sharding), which is critical for large-scale web applications.

Joins and Normalization: Relational databases use normalization and JOIN operations to eliminate data redundancy. NoSQL databases often denormalize data, embedding related information within a single document to reduce query complexity and improve read performance.

b) MongoDB Query — Retrieve Documents Where Category is "Electronics"

Assume the following sample document structure in a collection named products:

```
{
  "_id": ObjectId("64a1b2c3d4e5f6g7h8i9j0k1"),
  "name": "Samsung Galaxy S24",
  "category": "Electronics",
  "price": 79999,
  "brand": "Samsung",
  "stock": 150,
  "ratings": { "average": 4.5, "count": 320 }
}
```

Basic Query:

```
db.products.find({ category: "Electronics" })
```

Query with Projection (retrieve only specific fields):

```
db.products.find(
  { category: "Electronics" },
  { name: 1, price: 1, brand: 1, _id: 0 }
)
```

Query with Sorting and Limit (top 10 products by price descending):

```
db.products.find({ category: "Electronics" })
          .sort({ price: -1 })
          .limit(10)
```

Query with Additional Filter (Electronics under ₹50,000 with rating ≥ 4):

```
db.products.find({
  category: "Electronics",
```

```
  price: { $lt: 50000 },
  "ratings.average": { $gte: 4 }
})
```

Using countDocuments (count total Electronics products):

```
db.products.countDocuments({ category: "Electronics" })
```

Explanation: In MongoDB, find() takes a filter document as its first argument. The field category: "Electronics" performs an exact match. Dot notation ("ratings.average") is used to query nested fields. Projection (second argument) with 1 includes a field and 0 excludes it.

c) NoSQL Document Model for an E-Commerce Platform

A well-designed MongoDB schema for an e-commerce platform embeds related data within documents while maintaining references where necessary to avoid data duplication. Below is the complete model.### Complete MongoDB Document Schemas

1. Products Collection (with embedded reviews):

```
{
  "_id": ObjectId("prod001"),
  "name": "Samsung Galaxy S24",
  "description": "Latest flagship smartphone with AI features",
  "category": "Electronics",
  "brand": "Samsung",
  "price": 79999,
  "stock_qty": 150,
  "images": ["img1.jpg", "img2.jpg"],
  "specifications": {
    "RAM": "8GB",
    "storage": "256GB",
    "battery": "4000mAh"
  },
  "reviews": [
    {
      "reviewer_id": ObjectId("cust001"),
      "rating": 5,
      "comment": "Excellent camera and performance.",
      "date": ISODate("2024-08-10")
    },
    {
      "reviewer_id": ObjectId("cust002"),
      "rating": 4,
      "comment": "Good value for money.",
```

```
      "date": ISODate("2024-08-15")
    }
  ]
}
```

2. Customers Collection (with embedded addresses):

```
{
  "_id": ObjectId("cust001"),
  "name": "Ravi Kumar",
  "email": "ravi.kumar@email.com",
  "phone": "9876543210",
  "password_hash": "$2b$12$abcd...",
  "created_at": ISODate("2023-01-15"),
  "loyalty_points": 450,
  "addresses": [
    {
      "type": "home",
      "street": "12, MG Road",
      "city": "Guntur",
      "state": "Andhra Pradesh",
      "pincode": "522001"
    }
  ],
  "order_ids": [ObjectId("ord001"), ObjectId("ord002")]
}
```

3. Orders Collection (with embedded items and shipping):

```
{
  "_id": ObjectId("ord001"),
  "customer_id": ObjectId("cust001"),
  "order_date": ISODate("2024-09-01"),
  "status": "Delivered",
  "payment_method": "UPI",
  "total_amount": 84998,
  "items": [
    {
      "product_id": ObjectId("prod001"),
      "product_name": "Samsung Galaxy S24",
      "qty": 1,
      "unit_price": 79999,
      "subtotal": 79999
    },
    {
      "product_id": ObjectId("prod045"),
```

```
      "product_name": "Phone Case",
      "qty": 2,
      "unit_price": 2499,
      "subtotal": 4998
    }
  ],
  "shipping": {
    "address": {
      "street": "12, MG Road",
      "city": "Guntur",
      "pincode": "522001"
    },
    "courier": "Delhivery",
    "tracking_id": "DLV123456",
    "estimated_delivery": ISODate("2024-09-05")
  }
}
```

Design Decisions Explained

Embedding vs. Referencing is the central design choice in document databases. Reviews are embedded within the Products document because they are always accessed together with the product and have a one-to-many relationship bounded by the product. Order items are embedded within Orders because they are read atomically — retrieving an order always requires all its items. The product_name and unit_price are intentionally duplicated inside order items to preserve the historical record even if the product's price changes later.

Customer to Order relationship uses references (order_ids array in the Customer document pointing to _id values in the Orders collection) because a customer can have many orders over time, and orders are large documents that would bloat the customer record if embedded.

Indexes to consider for performance: { category: 1 } on Products for category-based filtering; { customer_id: 1 } on Orders for fast order history retrieval; { "reviews.reviewer_id": 1 } on Products for review lookups; and a compound index { category: 1, price: -1 } for filtered, sorted product listings.

Question 8:

a)

Relational Databases store data in the form of **tables** consisting of rows and columns. Each table has a fixed structure (schema), and relationships between tables are maintained using keys (primary key, foreign key). These databases are suitable for **structured data** and applications like banking and student records.

NoSQL Databases store data in a **flexible format** such as documents, key-value pairs, graphs, or columns. They do not require a fixed schema and are useful for handling **large and unstructured data** (Big Data), such as social media or e-commerce applications.

b)

Feature	Relational Databases	NoSQL Databases
Data Storage	Tables (rows & columns)	Documents (JSON-like)
Schema	Fixed schema	Flexible schema
Scalability	Vertical scaling	Horizontal scaling
Example	MySQL, Oracle	MongoDB

c)

i) Product Document Design

```
{
  "product_id": "P101",
  "name": "Smartphone",
  "price": 20000,
  "category": "Electronics"
}
```

ii) MongoDB Query

```
db.products.find({ category: "Electronics" })
```

This query retrieves all products belonging to the **Electronics** category.

iii) Extended Design (Orders and Reviews)

🛒 Orders Collection

```
{
  "order_id": "O1001",
  "customer_name": "Ravi",
  "items": [
    {
      "product_id": "P101",
      "name": "Smartphone",
      "quantity": 1
    }
  ],
  "total_amount": 20000
}
```

⭐ Reviews Collection

```
{
  "review_id": "R201",
  "product_id": "P101",
  "customer_name": "Ravi",
  "rating": 5,
  "comment": "Excellent product"
}
```